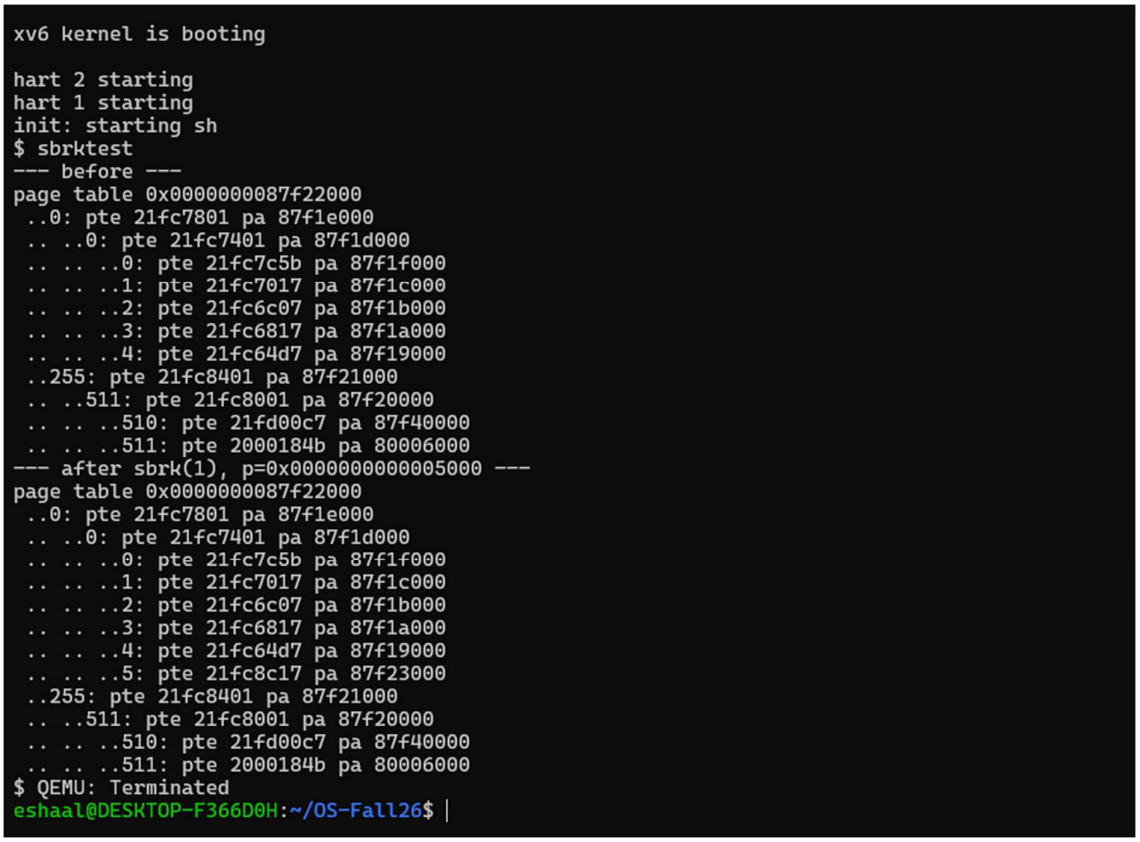
- kernel/defs.h: void vmprint(pagetable_t);
- kernel/syscall.h: #define SYS_pgprint 22
- kernel/syscall.c: add extern uint64 sys_pgprint(void); and [SYS_pgprint] sys_pgprint,
- kernel/sysproc.c: uint64 sys_pgprint(void){ vmprint(myproc()->pagetable); return 0; }
- user/usys.pl: entry("pgprint");
- user/user.h: int pgprint(void);
- Makefile: add $U/_sbrktest\ to UPROGS



1. How much space has the kernel allocated?

Exactly one 4096-byte page. Comparing the page table before and after sbrk(1), the only change is a new level-0 entry, index 5, covering VA 0x5000 (the old p->sz, which sbrk returned). No new page-table pages were allocated, because index 5 fits in the existing level-0 table. p->sz grows by only 1 byte, but growproc calls uvmalloc, which allocates and zeroes whole pages, so all of 0x5000 to 0x5fff is mapped and accessible.

2. What does the new PTE contain?

The new PTE is 0x21fc8c17.

- Flags (low 10 bits) are 0x17 = PTE_V | PTE_R | PTE_W | PTE_U (binary 0b0000010111): valid, readable, writable, user-accessible.
- PTE_X is not set, so the page is data and not executable.
- PTE_A (0x40) and PTE_D (0x80) are clear, because the page has not been accessed yet.
- PPN = 0x21fc8c17 >> 10 = 0x87f23, so the physical address is 0x87f23000, matching the printed pa.

Rest of the layout (from the “before” dump):

- Level-0 entries 0 to 4 are the program’s pages. Entry 0 (0x5b = V|R|X|U|A) is text. Entry 1 (0x17) is data. Entry 2 (0x07) has no PTE_U, so it is the stack guard page. Entries 3 and 4 (0x17, 0xd7) are the user stack. Entry 4 has A and D set because it has been written.
- The top of the address space holds the trapframe (index 510, 0xc7, no PTE_U) and the trampoline (index 511, 0x4b = V|R|X|A, no PTE_U).

Takeaway: sbrk allocates eagerly, in whole pages, at the time of the call. This is the behavior the lazy-allocation lab changes.

```
  GNU nano 7.2                                kernel/kalloc.c *
} kmem;

struct { struct spinlock lock; int cnt[PHYSTOP/PGSIZE]; } ref;
#define IDX(pa) ((uint64)(pa)/PGSIZE)

void kinit() {
  initlock(&ref.lock, "ref");
  initlock(&kmem.lock, "kmem");
  freerange(end, (void*)PHYSTOP);
}

void freerange(void *pa_start, void *pa_end) {
  char *p = (char*)PGROUNDUP((uint64)pa_start);
  for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE){
    ref.cnt[IDX(p)] = 1;       // so kfree below brings it to 0
    kfree(p);
  }
}

// Free the page of physical memory pointed at by pa,
// which normally should have been returned by a
// call to kalloc().  (The exception is when
// initializing the allocator; see kinit above.)
void
kfree(void *pa)
{
  struct run *r;

  if (((uint64)pa % PGSIZE) != 0 || (char *)pa < end || (uint64)pa >= PHYSTOP)
   panic("kfree");
  acquire(&ref.lock);
  if(--ref.cnt[IDX(pa)] > 0){ release(&ref.lock); return; }
  release(&ref.lock);

  // Fill with junk to catch dangling refs.
  memset(pa, 1, PGSIZE);

  r = (struct run *)pa;

  acquire(&kmem.lock);
  r->next = kmem.freelist;
  kmem.freelist = r;
  release(&kmem.lock);
}
```

What was added or changed:

1. kernel/riscv.h: #define PTE_COW (1L << 8)
2. kernel/kalloc.c:
    - ref struct with a lock and cnt[PHYSTOP/PGSIZE], plus the IDX(pa) macro
    - kinit initializes the ref lock
    - freerange sets cnt = 1 before calling kfree
    - kfree decrements the count and only frees the page when it reaches 0
    - kalloc sets cnt = 1 for the new page
    - new functions krefinc() and krefcnt()
3. kernel/vm.c:

    - uvmcopy no longer copies pages. It clears PTE_W, sets PTE_COW, maps the same pa in the child, and calls krefinc.
    - new cowalloc(): if the refcount is 1 it just restores PTE_W; otherwise it kalloc's, copies the page, remaps with PTE_W and without PTE_COW, and kfree's the old page.
    - copyout calls cowalloc when the destination PTE has PTE_COW
4. kernel/trap.c, usertrap: new branch else if(r_scause() == 15 && cowalloc(p->pagetable, r_stval()) == 0), placed before the vmfault branch and the final else
5. kernel/defs.h: prototypes for cowalloc, krefinc, krefcnt

Build errors we hit and the fixes (this tree differs from stock xv6):

- Missing semicolon on the vmprint prototype in defs.h. Added the ;.
- A stray printf prototype glued onto the either_copyout line in defs.h. Removed it.
- This tree uses printk, not printf. Changed the calls in vm.c to printk.
- printk has a format attribute, so %p on uint64 fails. Use %lx for pte and pa.
- krefinc/krefcnt/cowalloc were undeclared in vm.c. Added prototypes to defs.h.
- vmfault takes 4 args in this tree: vmfault(p->pagetable, p->sz, r_stval(), (r_scause() == 13) ? 1 : 0)